Assignment 3 – Part 1

Question 1.1

1

$$\{f : (T_1 \to T_2),\ g : (T_1 \to T_2),\ a : T_1\} \vdash (f\ (g\ a)) : T_2$$

False.

Since $g : T_1 \to T_2$ and $a : T_1$, we get $(g\ a) : T_2$. But f expects an argument of type T_1, not T_2.

2

$$\{f : (T_1 * T_2 \to T_3)\} \vdash (\lambda(x)\ (f\ x\ y)) : (T_2 \to T_3)$$

False.

The variable y is not in the environment, so the expression cannot be typed. Also, x is used as the first argument of f, so the lambda should have type $T_1 \to T_3$, not $T_2 \to T_3$.

3

$$\{f : (T_1 * T_2 \to T_3),\ y : T_2\} \vdash (\lambda(x)\ (f\ x\ y)) : (T_1 \to T_3)$$

True.

Question 1.2

1

Expression:

$$(\lambda(f1\ g1\ x1\ y1)\ (f1\ (g1\ y1\ x1)\ (g1\ x1\ y1)))$$

Pool

$(\lambda(f1\ g1\ x1\ y1)\ (f1\ (g1\ y1\ x1)\ (g1\ x1\ y1)))$	T_0
$(f1\ (g1\ y1\ x1)\ (g1\ x1\ y1))$	T_1
$f1$	T_{f1}
$(g1\ y1\ x1)$	T_2
$g1$	T_{g1}
$y1$	T_{y1}
$x1$	T_{x1}
$(g1\ x1\ y1)$	T_3

Equations

For the lambda expression:

$$T_0 = [T_{f1} * T_{g1} * T_{x1} * T_{y1} \to T_1]$$

For the outer application:

$$T_{f1} = [T_2 * T_3 \to T_1]$$

For the first application of $g1$:

$$T_{g1} = [T_{y1} * T_{x1} \to T_2]$$

For the second application of $g1$:

$$T_{g1} = [T_{x1} * T_{y1} \to T_3]$$

Unification

We have two equations for T_{g1}:

$$T_{g1} = [T_{y1} * T_{x1} \to T_2]$$

and:

$$T_{g1} = [T_{x1} * T_{y1} \to T_3]$$

Therefore, the unification algorithm needs to solve:

$$[T_{y1} * T_{x1} \to T_2] = [T_{x1} * T_{y1} \to T_3]$$

So we get:

$$T_{y1} = T_{x1}$$

and:

$$T_2 = T_3$$

The inference succeeds.
The final substitution is:

$$\{T_{y1} = T_{x1},\ T_3 = T_2\}$$

After applying this substitution:

$$T_{g1} = [T_{x1} * T_{x1} \to T_2]$$

and:

$$T_{f1} = [T_2 * T_2 \to T_1]$$

Therefore, the type of the whole expression is:

$$\boxed{T_0 = [[T_2 * T_2 \to T_1] * [T_{x1} * T_{x1} \to T_2] * T_{x1} * T_{x1} \to T_1]}$$

2

Expression:

$$((\lambda(f1)\ (\lambda(x1)\ (f1\ x1\ x1)))\ (\lambda(y1)\ y1))$$

Pool

$((\lambda(f1)\ (\lambda(x1)\ (f1\ x1\ x1)))\ (\lambda(y1)\ y1))$	T_0
$(\lambda(f1)\ (\lambda(x1)\ (f1\ x1\ x1)))$	T_1
$f1$	T_{f1}
$(\lambda(x1)\ (f1\ x1\ x1))$	T_2
$x1$	T_{x1}
$(f1\ x1\ x1)$	T_3
$(\lambda(y1)\ y1)$	T_4
$y1$	T_{y1}

Equations

For the application:

$$(f1\ x1\ x1)$$

the function $f1$ receives two arguments. Both arguments are $x1$, so both have type T_{x1}. The result has type T_3. Therefore:

$$T_{f1} = [T_{x1} * T_{x1} \rightarrow T_3]$$

For the inner lambda:

$$T_2 = [T_{x1} \rightarrow T_3]$$

For the outer lambda:

$$T_1 = [T_{f1} \rightarrow T_2]$$

For the identity function:

$$T_4 = [T_{y1} \rightarrow T_{y1}]$$

For the full application:

$$T_1 = [T_4 \rightarrow T_0]$$

Unification

We have:

$$T_1 = [T_{f1} \rightarrow T_2]$$

and:

$$T_1 = [T_4 \rightarrow T_0]$$

Therefore, the unification algorithm gets:

$$[T_{f1} \rightarrow T_2] = [T_4 \rightarrow T_0]$$

So it must unify:

$$T_{f1} = T_4$$

Now, from the equations above:

$$T_{f1} = [T_{x1} * T_{x1} \rightarrow T_3]$$

and:

$$T_4 = [T_{y1} \to T_{y1}]$$

Therefore, the algorithm tries to solve:

$$[T_{x1} * T_{x1} \to T_3] = [T_{y1} \to T_{y1}]$$

This fails because the left side is a function type with two arguments, while the right side is a function type with one argument.

Therefore, the inference fails.

The error is caused by trying to unify a binary function type with a unary function type:

$$\boxed{[T_{x1} * T_{x1} \to T_3] = [T_{y1} \to T_{y1}]}$$

3

Expression:

$$((\lambda(f1)\ (\lambda(x1)\ ((f1\ x1)\ x1)))\ (\lambda(y1)\ y1))$$

Pool

$((\lambda(f1)\ (\lambda(x1)\ ((f1\ x1)\ x1)))\ (\lambda(y1)\ y1))$	T_0
$(\lambda(f1)\ (\lambda(x1)\ ((f1\ x1)\ x1)))$	T_1
$f1$	T_{f1}
$(\lambda(x1)\ ((f1\ x1)\ x1))$	T_2
$x1$	T_{x1}
$((f1\ x1)\ x1)$	T_3
$(f1\ x1)$	T_4
$(\lambda(y1)\ y1)$	T_5
$y1$	T_{y1}

Equations

For the application:

$$(f1\ x1)$$

we get:

$$T_{f1} = [T_{x1} \to T_4]$$

For the application:

$$((f1\ x1)\ x1)$$

we get:

$$T_4 = [T_{x1} \to T_3]$$

For the inner lambda:

$$T_2 = [T_{x1} \to T_3]$$

For the outer lambda:

$$T_1 = [T_{f1} \to T_2]$$

For the identity function:

$$T_5 = [T_{y1} \to T_{y1}]$$

For the full application:

$$T_1 = [T_5 \to T_0]$$

Unification

We have:

$$T_1 = [T_{f1} \to T_2]$$

and:

$$T_1 = [T_5 \to T_0]$$

Therefore:

$$T_{f1} = T_5$$

But:

$$T_5 = [T_{y1} \to T_{y1}]$$

and:

$$T_{f1} = [T_{x1} \to T_4]$$

Therefore:

$$[T_{x1} \to T_4] = [T_{y1} \to T_{y1}]$$

So:

$$T_{x1} = T_{y1}$$

and:

$$T_4 = T_{y1}$$

Since $T_{x1} = T_{y1}$, we get:

$$T_4 = T_{x1}$$

But we also have:

$$T_4 = [T_{x1} \to T_3]$$

Therefore, the algorithm tries to solve:

$$T_{x1} = [T_{x1} \to T_3]$$

This fails because T_{x1} occurs inside the type it is being assigned to. This is an occurrence-check error.

Therefore, the inference fails.

The error is caused by the equation:

$$\boxed{T_{x1} = [T_{x1} \to T_3]}$$

Question 1.3

Expression:

$$(\lambda(f)\ (f\ (right\ (\lambda(s)\ (f\ (left\ s))))))$$

Pool

We assign a fresh type variable to each sub-expression:

$$\begin{array}{ll}
(\lambda(f)\ (f\ (right\ (\lambda(s)\ (f\ (left\ s)))))) & T_0 \\
(f\ (right\ (\lambda(s)\ (f\ (left\ s))))) & T_1 \\
f & T_f \\
(right\ (\lambda(s)\ (f\ (left\ s)))) & T_2 \\
(\lambda(s)\ (f\ (left\ s))) & T_3 \\
(f\ (left\ s)) & T_4 \\
(left\ s) & T_5 \\
s & T_s
\end{array}$$

We also introduce fresh type variables for the unknown sides of the sum types:

$$T_6,\ T_7$$

Equations

For the expression:

$$(left\ s)$$

we use the rule for $left$. Since $s : T_s$, the expression $(left\ s)$ has type $T_s + T_6$, where T_6 is fresh. Therefore:

$$T_5 = T_s + T_6$$

For the expression:

$$(f\ (left\ s))$$

the argument has type T_5, and the result has type T_4. Therefore:

$$T_f = [T_5 \to T_4]$$

For the lambda expression:

$$(\lambda(s)\ (f\ (left\ s)))$$

the parameter s has type T_s, and the body has type T_4. Therefore:

$$T_3 = [T_s \to T_4]$$

For the expression:

$$(right\ (\lambda(s)\ (f\ (left\ s))))$$

we use the rule for *right*. The inner expression has type T_3, so the whole expression has type $T_7 + T_3$, where T_7 is fresh. Therefore:

$$T_2 = T_7 + T_3$$

For the outer application:

$$(f\ (right\ (\lambda(s)\ (f\ (left\ s)))))$$

the argument has type T_2, and the result has type T_1. Therefore:

$$T_f = [T_2 \to T_1]$$

For the whole lambda expression:

$$(\lambda(f)\ (f\ (right\ (\lambda(s)\ (f\ (left\ s))))))$$

the parameter f has type T_f, and the body has type T_1. Therefore:

$$T_0 = [T_f \to T_1]$$

Unification

We have two equations for T_f:

$$T_f = [T_5 \to T_4]$$

and:

$$T_f = [T_2 \to T_1]$$

Therefore, the unification algorithm needs to solve:

$$[T_5 \to T_4] = [T_2 \to T_1]$$

So we get:

$$T_5 = T_2$$

and:

$$T_4 = T_1$$

Now we use the equations for T_5, T_2, and T_3:

$$T_5 = T_s + T_6$$

$$T_2 = T_7 + T_3$$

$$T_3 = [T_s \to T_4]$$

Since $T_5 = T_2$, we get:

$$T_s + T_6 = T_7 + T_3$$

Substituting $T_3 = [T_s \to T_4]$, we get:

$$T_s + T_6 = T_7 + [T_s \to T_4]$$

Now we unify the two sum types.
From the left sides of the sum types:

$$T_s = T_7$$

From the right sides of the sum types:

$$T_6 = [T_s \to T_4]$$

Also, from before:

$$T_1 = T_4$$

Therefore, the final substitution is:

$$\{T_7 = T_s,\ T_6 = [T_s \to T_4],\ T_1 = T_4\}$$

The inference succeeds.
After applying the substitution, we get:

$$T_f = [T_s + [T_s \to T_4] \to T_4]$$

Finally, since:

$$T_0 = [T_f \to T_1]$$

and $T_1 = T_4$, the type of the whole expression is:

$$T_0 = [[T_s + [T_s \to T_4] \to T_4] \to T_4]$$

Therefore:

$$\boxed{[[T_s + [T_s \to T_4] \to T_4] \to T_4]}$$